DAILY TASK MANAGER

A
Data Structures Course Project Report

By

Team No:

AKSHAYA BHOOMIREDDY 1602-24-737-001
DESHITHA SREERAMULU 1602-24-737-012

Department of Information Technology

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University)

Ibrahimbagh,

Hyderabad-500 031

2025

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University)

Hyderabad-500 031

Department of Information Technology

DECLARATION BY THE CANDIDATE

We, **AKSHAYA BHOOMIREDDY , DESHITHA SRRERAMULU** bearing hall ticket numbers, **1602-24-737-001** and **1602-24-737-012**, hereby declare that the project report entitled **"DAILY TASK MANAGER"**is submitted in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Information Technology**

This is a record of project work carried out by us and submitted in the form of a report.

AKSHAYA BHOOMIREDDY
1602-24-737-001
DESHITHA SREERAMULU
1602-24-737-012

SIREESHA PANYAM **Dr. K. Ram Mohan Rao**

ACKNOWLEDGMENT

We extend our sincere thanks to Dr. S. V. Ramana,Principal, Vasavi College of Engineering for his encouragement.

We express our sincere gratitude to Dr. K. Ram Mohan Rao, Professor & Head, Department of Information Technology, Vasavi College of Engineering.

We also want to thank and convey our gratitude towards our DS Faculty, SIREESHA PANYAM for guiding us in understanding the process of project development and giving us timely suggestions at every phase.

ABSTRACT

Effective daily task management and personal organization remain critical challenges for individuals, often leading to missed priorities and reduced productivity. This necessity defines the Problem Statement: the need for a simple, efficient, and accessible tool to organize routine activities. This problem is important because poor organization directly impacts success in both academic and professional life. The proposed solution is the Daily Task Manager, a console-based application developed in the C programming language that uses fundamental data structures to manage tasks. Specifically, a Queue is implemented to enforce a disciplined, chronological (First-In, First-Out, or FIFO) order for task completion, ensuring that pending activities are addressed sequentially. Concurrently, a Stack is employed to offer a crucial 'undo' feature (Last-In, First-Out, or LIFO), allowing the user to reverse the most recent task addition instantly, thereby enhancing user-friendliness and error correction. The benefit of this proposed work is a lightweight, command-line utility that promotes better habits through structured workflow and positive reinforcement (via motivational quotes). The research community benefits by having a practical, demonstrated example of how the core characteristics of the Queue and Stack data structures are strategically combined to model two distinct real-world requirements—sequencing and reversal—within a single application.

Background and Domain

In the modern digital landscape, individuals operate under a constant loads of information and an increasing number of responsibilities, making effective **personal organization** and **time management** essential for maintaining productivity and achieving goals. Task management software, in general, has evolved from simple digital checklists into sophisticated systems that serve as an **external cognitive aid**, successfully offloading the mental burden of remembering deadlines and priorities. This technological necessity ensures that users can transition from being reactive to proactive, dedicating their limited mental resources to task execution rather than tracking and recall, thereby addressing the complexities of managing numerous commitments in a multi-platform environment

Problem Statement

Despite the availability of numerous task management applications, there remains a need for a **simple, educational, and structurally sound tool** that explicitly demonstrates the application of core computer science principles. The current problem is the lack of a console-based utility that precisely models two key behavioral requirements: enforced **sequential completion** and **immediate reversal (undo)**. Developing such a tool in C allows for a clear, demonstrable link between abstract data structures and practical software features, serving as an effective educational resource while providing a reliable, low-overhead solution for managing daily to-dos.

Project Objectives

The primary goal of the **Daily Task Manager** project is to deliver a functional console application that achieves three key structural and operational objectives. First, it aims to **Implement a Queue** data structure to guarantee a **FIFO (First-In, First-Out)** execution flow, ensuring tasks are completed in the order they were assigned. Second, the project must **Implement a Stack** data structure to facilitate a **LIFO (Last-In, First-Out)** 'undo' mechanism, allowing users to instantly reverse the last task addition. Finally, the project aims to integrate these data structures into a cohesive, **menu-driven user interface** using the C programming language, ensuring efficient use of dynamically allocated memory through linked lists

Necessity of the Problem Statement

The necessity of addressing this problem stems directly from the requirement to bridge the gap between theoretical computer science concepts and practical application development. While high-level programming languages often abstract away the complexities of data management, the Structured Programming for Problem Solving (SPPS) curriculum mandates a fundamental understanding of how data structures function at the implementation level. Specifically, this project is needed to showcase how the core behaviors of the Queue (First-In, First-Out) and the Stack (Last-In, First-Out) are not merely abstract concepts, but are the logical foundation for creating functional features like task sequencing and command reversal in real-world applications. By forcing the use of these specific structures, the project ensures the learner masters dynamic memory allocation and pointer manipulation in C, which are critical skills often overlooked in modern, garbage-collected environments.

Need or Purpose

The fundamental purpose of the Daily Task Manager is twofold: pedagogical and practical. Pedagogically, it serves as a robust demonstration of dynamic memory management in C, providing a clear model for implementing abstract data types like Queues and Stacks using linked lists. Practically, it addresses the need for a simple, reliable command-line tool that students and developers can use without external dependencies or heavy graphical interfaces. This approach provides an efficient way to manage daily to-dos while adhering to the design principle of structural clarity. By demonstrating the efficiency of $O(1)$ operations for both

adding and completing tasks, the project validates the choice of linked list-based data structures for dynamic, high-frequency operations.

Justification of chosen Data Structure 1: Queue

Role in Project: Used for the main task list (To-Do List).

Justification: The Queue enforces a FIFO (First-In, First-Out) order. Daily tasks are typically done sequentially; the task added first should ideally be the task completed first. Using a Queue ensures that the Complete Task function always targets the oldest, most pressing task, promoting structured execution

Justification of chosen Data Structure 2: Stack

Role in Project: Used for the Undo Last Task Added feature[2].

Justification: The Stack enforces a LIFO (Last-In,First-Out) order. In an undo feature, the most recent action (adding a task) is the first action that needs to be reversed. The Stack provides this immediate reversal capability efficiently O(1) time complexity for push/pop.

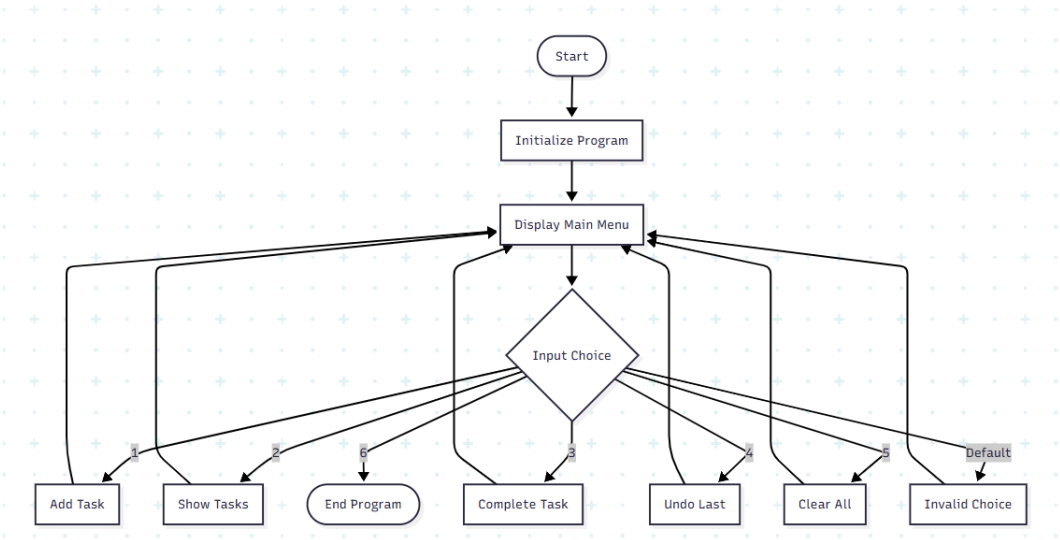
Justification of chosen Data Structure 3: Linked List (Underlying Structure)

Role in Project: Used to dynamically implement both the Queue and the Stack.

Justification: Since the number of daily tasks is unpredictable, a linked list is preferred over a fixed-size array. It allows the Queue and Stack to grow or shrink

dynamically as tasks are added or completed, preventing overflow and ensuring efficient use of memory

Design - Flowchart describing the Overall Project



Implementation

Software and Hardware Requirements

The Daily Task Manager is built using the **C Programming Language**, requiring minimal resources and relying on standard development tools. The core software environment necessitates a robust C Compiler, such as the **GNU C Compiler (GCC)**, to translate the source code into an executable application, along with a basic **Text Editor** for development. The application is designed to be platform-independent at the source code level, allowing it to run on any major operating system, including **Windows, Linux, and macOS**. Hardware requirements are minimal, demanding only a functional machine with a **Processor** equivalent to an Intel Core i3 or higher, and at least **2 GB of RAM**, ensuring that the console-based program executes quickly and efficiently on virtually any modern computer system.

Module 1 code explanation: Queue Operations

The task list is managed by the **Queue** data structure, which is built upon a singly linked list. This implementation begins with the struct Task, which holds the task name and a pointer (next) to the subsequent task in the list, and the struct Queue, which maintains pointers to both the front and the rear nodes. The core operation, enqueue(), efficiently adds a new task by first dynamically allocating memory for a new node, copying the task name, and then assigning this new node to the current rear's next pointer before updating the rear pointer to the new node, ensuring a constant **O(1) time complexity**. Conversely, the dequeue() operation facilitates task completion by checking if front is NULL, saving the front node to a temporary pointer, advancing the front pointer to the next node, and finally freeing the memory of the completed task, which also executes in **O(1) constant time**.

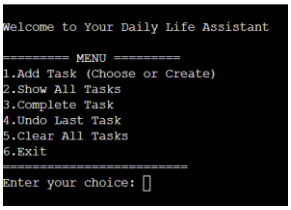
```
struct Queue {
   struct Task *front;
   struct Task *rear;
};
```

Module 2 code explanation: Stack Operations

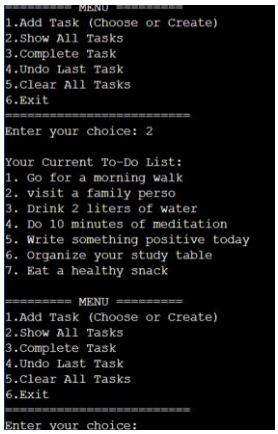
The **Undo** feature is handled by the **Stack** data structure, which is governed by the struct Stack containing a single pointer, top, pointing to the last node added. The push() operation, called immediately after enqueue(), creates a new node, sets its next pointer to the current top, and then updates the top pointer to the new node, effectively performing insertion at the beginning of the list (LIFO). The corresponding pop() operation enables the undo functionality by ensuring the stack is not empty, saving the top node, advancing the top pointer down the list, and then freeing the memory of the undone task. Because both push and pop only involve modifying pointers at one end of the list, both operations are executed in **O(1) constant time**

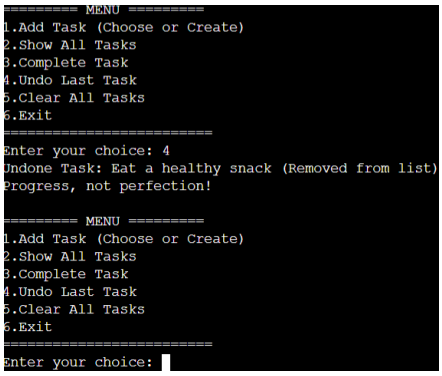
```
struct Stack {
   struct Task *top;
};
```

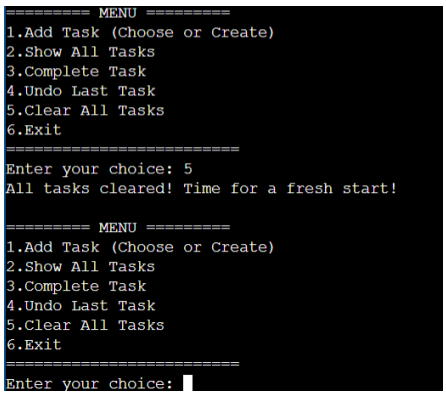
Output Screen Shots

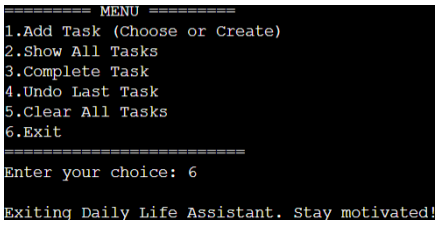



Welcome to Your Daily Life Assistant
========= MENU =========
1.Add Task (Choose or Create)
2.Show All Tasks
3.Complete Task
4.Undo Last Task
5.Clear All Tasks
6.Exit
==========
Enter your choice: 1
Choose a Task from List or Enter Your Own:
1. Go for a morning walk
2. Read 10 pages of a book
3. Drink 2 liters of water
4. Do 10 minutes of meditation
5. Listen to a relaxing song
6. Eat a healthy snack
7. Write something positive today
8. Call or message a loved one
9. Organize your study table
10. Take a short power nap
11.Enter My Own Task
Enter your choice (1-11): 1
Task Added: Go for a morning walk
Believe in yourself!
========= MENU =========
1.Add Task (Choose or Create)
2.Show All Tasks
3.Complete Task
4.Undo Last Task
5.Clear All Tasks
6.Exit
==========
Enter your choice:


========= MENU =========
1.Add Task (Choose or Create)
2.Show All Tasks
3.Complete Task
4.Undo Last Task
5.Clear All Tasks
6.Exit
==========
Enter your choice: 2
Your Current To-Do List:
1. Go for a morning walk
2. visit a family perso
3. Drink 2 liters of water
4. Do 10 minutes of meditation
5. Write something positive today
6. Organize your study table
7. Eat a healthy snack
========= MENU =========
1.Add Task (Choose or Create)
2.Show All Tasks
3.Complete Task
4.Undo Last Task
5.Clear All Tasks
6.Exit
==========
Enter your choice:


========= MENU =========
1.Add Task (Choose or Create)
2.Show All Tasks
3.Complete Task
4.Undo Last Task
5.Clear All Tasks
6.Exit
==========
Enter your choice: 4
Undone Task: Eat a healthy snack (Removed from list)
Progress, not perfection!
========= MENU =========
1.Add Task (Choose or Create)
2.Show All Tasks
3.Complete Task
4.Undo Last Task
5.Clear All Tasks
6.Exit
==========
Enter your choice:


========= MENU =========
1.Add Task (Choose or Create)
2.Show All Tasks
3.Complete Task
4.Undo Last Task
5.Clear All Tasks
6.Exit
==========
Enter your choice: 5
All tasks cleared! Time for a fresh start!
========= MENU =========
1.Add Task (Choose or Create)
2.Show All Tasks
3.Complete Task
4.Undo Last Task
5.Clear All Tasks
6.Exit
==========
Enter your choice:


========= MENU =========
1.Add Task (Choose or Create)
2.Show All Tasks
3.Complete Task
4.Undo Last Task
5.Clear All Tasks
6.Exit
==========
Enter your choice: 6
Exiting Daily Life Assistant. Stay motivated!

Future Scope

Persistence: Implement file handling (saving/loading tasks to a file) so tasks persist after the program exits.

Priority Queue: Introduce task priority levels (e.g., High, Medium, Low) and use a Priority Queue data structure instead of a simple Queue.

Search/Edit: Add functionality to search for or edit existing tasks within the list.

CONCLUSION

The Daily Task Manager successfully demonstrates the practical application of fundamental data structures, Queue and Stack, in building a simple, yet functional, task management system. The Queue effectively enforces a disciplined, chronological (FIFO) approach to task completion, while the Stack provides a quick-access, immediate 'undo' mechanism for error correction (LIFO). The implementation in C, using linked lists for dynamic memory allocation, ensures efficient use of resources. The addition of motivational quotes enhances the user experience, transforming a simple data structure exercise into a useful "Daily Life Assistant." The project serves as a clear illustration of how abstract data structure concepts translate into real-world software features.

REFERENCES

Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.) Prentice Hall.